Name: **Aayush Patil**
Class: **D20A**
Roll No: **56**

Lab - 2: **Experiment No: 2** **Dated:**
Aim: Create a Blockchain using Python

Theory:
Blockchain is a distributed and immutable ledger that records data in a sequence of linked blocks. Each block contains a timestamp, proof (from Proof-of-Work), and the hash of the previous block, ensuring integrity and security.

In this program:

1. **Block Creation** – A block is generated using create_block() with a proof and previous hash.
2. **Proof-of-Work (PoW)** – Implemented to mine a block by solving a cryptographic puzzle (finding a hash with leading zeros).
3. **Hashing** – SHA-256 algorithm ensures data immutability.
4. **Validation** – is_chain_valid() checks that every block correctly references the previous hash and satisfies PoW conditions.
5. **Flask Web API** – Routes /mine_block, /get_chain, and /is_valid allow mining, fetching the chain, and verifying blockchain integrity through a browser or API client.

This program runs on a local server and simulates mining a blockchain without a network of nodes, making it an ideal introductory model to understand blockchain basics.

1. What is a Blockchain?
2. Process of Mining
3. How to check the validity of blocks in a Blockchain

Code:

1. Importing Required Libraries

- **datetime** → to timestamp each block
- **hashlib** → to generate SHA-256 hashes
- **json** → to encode blocks as JSON for hashing
- **Flask** → to run the blockchain API server

---

2. Blockchain Class – Core Logic

This contains everything needed to:

- create blocks
- store the chain
- implement Proof of Work
- validate chain integrity

---

3. Flask Web API

Provides endpoints:

- /mine_block → mine a new block
- /get_chain → fetch full blockchain
- /is_valid → verify blockchain

[1] ✓ 0s

```
import datetime
import hashlib
import json
```

[2] ✓ 0s

```
class Blockchain:
    def __init__(self):
        self.chain = []
        self.create_block(proof = 1, previous_hash = '0')

    def create_block(self, proof, previous_hash):
        block = {'index': len(self.chain) + 1,
                 'timestamp': str(datetime.datetime.now()),
                 'proof': proof,
                 'previous_hash': previous_hash}
        self.chain.append(block)
        return block

    def get_previous_block(self):
        return self.chain[-1]

    def proof_of_work(self, previous_proof):
        new_proof = 1
        check_proof = False
        while check_proof is False:
            hash_operation = hashlib.sha256(str(new_proof**2 - previous_proof**2).encode()).hexdigest()
            if hash_operation[:4] == '0000':
                check_proof = True
            else:
                new_proof += 1
        return new_proof

    def hash(self, block):
        encoded_block = json.dumps(block, sort_keys = True).encode()
        return hashlib.sha256(encoded_block).hexdigest()

    def is_chain_valid(self, chain):
        previous_block = chain[0]
        block_index = 1
        while block_index < len(chain):
            block = chain[block_index]
            if block['previous_hash'] != self.hash(previous_block):
                return False
```

```
            previous_block = block
            block_index += 1
        return True
```

```
# Creating a Blockchain
blockchain = Blockchain()
```

```
# Mining a new block
def mine_block():
    previous_block = blockchain.get_previous_block()
    previous_proof = previous_block['proof']
    proof = blockchain.proof_of_work(previous_proof)
    previous_hash = blockchain.hash(previous_block)
    block = blockchain.create_block(proof, previous_hash)
    response = {'message': 'Congratulations Yogita  Patil, you just mined a block!',
                'index': block['index'],
                'timestamp': block['timestamp'],
                'proof': block['proof'],
                'previous_hash': block['previous_hash']}
    return response
```

```
# Getting the full Blockchain
def get_chain():
    response = {'chain': blockchain.chain,
                'length': len(blockchain.chain)}
    return response
```

```
# Checking if the Blockchain is valid
def is_valid():
    is_valid = blockchain.is_chain_valid(blockchain.chain)
    if is_valid:
        response = {'message': 'All good Yogita Patil. The Blockchain is valid.'}
    else:
        response = {'message': 'Houston, we have a problem. The Blockchain is not valid.'}
    return response
```

```
Sprint("Functions Menu:")
print("===============")
print("1. Mine a block")
print("2. Display the chain")
print("3. Check the validity of the chain")
choice = int(input("Enter your choice :"))
if choice == 1:
  print(mine_block())
elif choice == 2:
  print(get_chain())
elif choice == 3:
  print(is_valid())
else :
  print("Invalid choice")
```

```
Functions Menu:
===============
1. Mine a block
2. Display the chain
3. Check the validity of the chain
Enter your choice :2
{'chain': [{'index': 1, 'timestamp': '2026-08-20 03:27:09.696840', 'proof': 1, 'previous_hash': '0'}, {'index': 2, 'timestamp': '2026-08-20 03:27:29.679131', 'proof': 533,
```

```
Functions Menu:
===============
1. Mine a block
2. Display the chain
3. Check the validity of the chain
Enter your choice :1
{'message': 'Congratulations Yogita  Patil, you just mined a block!', 'index': 4, 'timestamp': '2026-08-20 03:38:48.303645', 'proof': 21391, 'previous_hash': 'f302bbd9f387327b6fe943d
```

```python
import datetime
import hashlib
import json

class Blockchain:
    def __init__(self):
        self.chain = []
        # Create genesis block
        self.create_block(proof = 1, previous_hash = '0')

    def create_block(self, proof, previous_hash):
        block = {
            'index': len(self.chain) + 1,
            'timestamp': str(datetime.datetime.now()),
            'proof': proof,
            'previous_hash': previous_hash
        }
        # Compute the current hash of the block structur4
        e and append it
        block['hash'] = self.hash(block)

        self.chain.append(block)
        return block

    def get_previous_block(self):
        return self.chain[-1]

    def proof_of_work(self, previous_proof):
        new_proof = 1
        check_proof = False
        while check_proof is False:
            hash_operation = hashlib.sha256(str(new_proof**2 - previous_proof**2).encode()
            if hash_operation[:4] == '0000':
                check_proof = True
            else:
                new_proof += 1
        return new_proof

    def hash(self, block):
        # Remove the 'hash' key temporarily during recalculation to avoid corrupting data
        block_copy = {k: v for k, v in block.items() if k != 'hash'}
        encoded_block = json.dumps(block_copy, sort_keys = True).encode()
        return hashlib.sha256(encoded_block).hexdigest()

    def is_chain_valid(self, chain):
        previous_block = chain[0]
        block_index = 1
        while block_index < len(chain):
            block = chain[block_index]

            if block['previous_hash'] != self.hash(previous_block):
                return False

            if block['hash'] != self.hash(block):
                return False

            previous_proof = previous_block['proof']
            proof = block['proof']
            hash_operation = hashlib.sha256(str(proof**2 - previous_proof**2).encode()).he
            if hash_operation[:4] != '0000':
                return False
```

```
            previous_block = block
            block_index += 1
        return True

# Initialize a single persistent blockchain instance
blockchain = Blockchain()

def mine_block():
    previous_block = blockchain.get_previous_block()
    previous_proof = previous_block['proof']
    proof = blockchain.proof_of_work(previous_proof)
    previous_hash = previous_block['hash'] # Dynamically uses the last block's current hash
    block = blockchain.create_block(proof, previous_hash)

    response = {
        'message': 'Congratulations Yogita Patil, you just mined a block!',
        'index': block['index'],
        'timestamp': block['timestamp'],
        'proof': block['proof'],
        'previous_hash': block['previous_hash'],
        'hash': block['hash']
    }
    return response

def get_chain():
    return {
        'chain': blockchain.chain,
        'length': len(blockchain.chain)
    }

def is_valid():
    if blockchain.is_chain_valid(blockchain.chain):
        return {'message': 'All good Yogita Patil. The Blockchain is valid.'}
    return {'message': 'Houston, we have a problem. The Blockchain is not valid.'}

# Continuous Execution Loop
while True:
    print("\nFunctions Menu:")
    print("===============")
    print("1. Mine a block")
    print("2. Display the chain")
    print("3. Check the validity of the chain")
    print("4. Exit program")

    try:
        choice = int(input("Enter your choice: "))
        if choice == 1:
            print("\nMining block... Please wait...")
            print(json.dumps(mine_block(), indent=4))
        elif choice == 2:
            print(json.dumps(get_chain(), indent=4))
        elif choice == 3:
            print(json.dumps(is_valid(), indent=4))
        elif choice == 4:
            print("Exiting application loop.")
            break
        else:
            print("Invalid choice! Select numbers between 1 and 4.")
    except ValueError:
        print("Please enter a valid integer choice.")
```

```
3. Check the validity of the chain
4. Exit program
Enter your choice: 1

Mining block... Please wait...
{
    "message": "Congratulations Yogita Patil, you just mined a block!",
    "index": 4,
    "timestamp": "2026-08-20 04:05:32.252753",
    "proof": 21391,
    "previous_hash": "a7360b366f0341f0bf04eec98f45833f62ed9081bf1db5767f60799f4b54efdf",
    "hash": "7f31f5e53bd5d1173fb2c06cd553a6c456ea1e994357b89de296f7eaf19b367c"
}

Functions Menu:
===============
1. Mine a block
2. Display the chain
3. Check the validity of the chain
4. Exit program
Enter your choice: 2
{
    "chain": [
        {
            "index": 1,
            "timestamp": "2026-08-20 04:05:02.001189",
            "proof": 1,
            "previous_hash": "0",
            "hash": "b28fb7ac70d655ff3d16f2097c8a96ce1a3990b47eafc846a7fdb8ac897003dc"
        },
        {
            "index": 2,
            "timestamp": "2026-08-20 04:05:08.639983",
            "proof": 533,
            "previous_hash": "b28fb7ac70d655ff3d16f2097c8a96ce1a3990b47eafc846a7fdb8ac897003dc",
            "hash": "9735c615852bd86201f178770802651b7f91821544f3771b59b8fc261402dd81"
        },
        {
            "index": 3,
            "timestamp": "2026-08-20 04:05:18.458258",
            "proof": 45293,
            "previous_hash": "9735c615852bd86201f178770802651b7f91821544f3771b59b8fc261402dd81",
            "hash": "a7360b366f0341f0bf04eec98f45833f62ed9081bf1db5767f60799f4b54efdf"
        },
        {
            "index": 4,
            "timestamp": "2026-08-20 04:05:32.252753",
            "proof": 21391,
            "previous_hash": "a7360b366f0341f0bf04eec98f45833f62ed9081bf1db5767f60799f4b54efdf",
            "hash": "7f31f5e53bd5d1173fb2c06cd553a6c456ea1e994357b89de296f7eaf19b367c"
        }
    ],
```

Conclusion:

We successfully created a basic blockchain using Python and Flask that supports block mining, chain retrieval, and validity checks. Proof-of-Work ensures security by making block creation computationally intensive, while cryptographic hashing guarantees immutability. The implementation demonstrates core blockchain principles—decentralization, immutability, and consensus—in a simplified environment, providing a strong foundation for building more advanced blockchain systems.